

Visual Computing

Janis Hutz

<https://janishutz.com>

September 26, 2026

1 Computer Vision

1.1 Digital Image

Def Image $f: \mathbb{R}^n \rightarrow S$, for greyscale $n = 2$, $S = \mathbb{R}^+$, in digital form: $I: \{1, \dots, X\} \times \{1, \dots, Y\} \rightarrow S$

Rem Pinhole size affects blur, **Lens Camera** as sol.

Rem Charge Coupled Device (CCD) sensor reads line-by-line (rolling shutter), then each line pixel-by-pixel

Def Sensor array is an array of *photosites* (bucket of el. charge $\propto$ light intensity), then read via ADC.

Rem Blooming Caused by photosite saturation

Def Dark Current CCDs give non-zero output in darkness, fluctuates randomly

Def CMOS Sensor Each photo sensor has own amplifier, **more noise, lower sensitivity**, “smart” pixels bc. CMOS

Def Sampling Repeated measurement on certain interval, **Reconstruction** is inverse. Issues: **undersampling** (lose information), **oversampling** (too large files).

Def Nyquist Frequency highest signal freq that can be accurately captured, half of sampling rate

Def Quantization Lossy discretization of analogue value, simple versions have $k = 2^b$ values for b bits

Rem Quantization “on” y -axis, Sampling “along” x -axis

Def Res. Geometric: PPI; Radiometric: Bits/Pixel

Def Signal-Noise-R. (SNR) Image quality index: $s = F \div \sigma$, with $F = \frac{1}{XY} \sum_{x=1}^X \sum_{y=1}^Y f(x, y)$.

Def Add. Gaussian Noise $I(x, y) = f(x, y) + c$, with $c \sim \mathcal{N}(0, \sigma^2)$, s.t. $p(c) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{c^2}{2\sigma^2}}$

Def Poisson Noise $p(k) = \frac{\lambda^k e^{-\lambda}}{k!}$

1.2 Filtering & Convolution

Def Moving Average New image G where $G[x, y]$ is average of surrounding pixels and $F[x, y]$

Def Linear Shift-Invariant Filter. New pixels lin. comb. of neighbours, *shift-invariant* = doing same for all pixels. Good for: smoothing, noise reduction, sharpening

Def Linear Operation L lin. if $L[\alpha I_1 + \beta I_2] = \alpha L[I_1] + \beta L[I_2]$, $I'_j = \sum_{i=1}^N \alpha_{ij} I_i$ for $j \in \{1, \dots, N\}$

Def Linear Filtering $I'(x, y) = \sum_{(i,j) \in N(x,y)} K(i, j) I(x + i, y + j)$ with I the input image and K the **kernel** of operation, $N(x, y)$ neighbourhood function for pixel (x, y) . Shift-Invariant if K doesn't depend on (x, y) (i.e. same weights everywhere)

Def Kernel Typically matrix, center of matrix is $(0, 0)$

Def Correlation $I'(x, y) = \sum_{j=-k}^k \sum_{i=-k}^k K(i, j) I(x + i, y + j)$

Def Conv. $I'(x, y) = \sum_{j=-k}^k \sum_{i=-k}^k K(-i, -j) I(x + i, y + j)$

Gen.: $g(x) = (f * h)(x) = \int_{-\infty}^{\infty} f(\tau) h(x - \tau) d\tau$, $h(\tau)$ kernel

Rem If $K(i, j) = K(-i, -j)$, then Convolution = Corr.

Rem Convolution is linear: $[(af + bg) * h](x) = a(f * h)(x) + b(g * h)(x)$, commutative: $(f * h)(x) = (h * f)(x)$, associative: $(f * g) * h = f * (g * h)$; Diff: $\frac{d}{dx} (f * h)(x) = f * h'$

Rem At edge of image, extrapolate, options: clip filter (black), wrap around, copy edge, reflect across edge

Def Gaussian Kernel $G_\sigma = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$. The smoothing amount depends on σ and window size.

Rem $g(x, y) = g(x)g(y)$, with $g(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2}{2\sigma^2}\right)$. Efficient implementation: First rows 1D, then cols 1D

1.3 Edge Detection

Def Scale Space Collection of img at diff scales & smooth.

Rem Finite diff. for $\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x+1, y) - f(x, y)}{1}$, filter: $[1, -1]$. Can use par der. for edge detection

Def Gradient $\nabla f = [\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}]$, dir of most rapid change.

Edge strength (magnitude): $M(x, y) = \|\nabla f\|$.

Noise makes edge detection hard, smooth first, then edge is peak in $\frac{\partial}{\partial x} (h * f)$. **Angle** $\alpha(x, y) = \arctan\left(\frac{\partial f}{\partial y} \div \frac{\partial f}{\partial x}\right)$

Thm $\frac{\partial}{\partial x} (h * f) = \left(\frac{\partial}{\partial x} h\right) * f$

Def Laplacian $\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$

Def LoG $\nabla^2 G(x, y) = \frac{1}{2\pi\sigma^2} \left(\frac{x^2+y^2-2\sigma^2}{\sigma^2}\right) \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$

Edge is at zero-crossing of $\left(\frac{\partial^2}{\partial^2 x} h\right) * f$. σ affects scale of edges detected (larger σ = stronger edges)

Def Canny E.D. **1** smooth w/ Gaussian, **2** gradient magnitude & angle, **3** Nonmaxima suppression to gradient mag. image, **4** double thresholding to detect strong and weak edge pixels, **5** reject weak edge pixels not connected with strong edge pixels

Def Non-Max-Suppression Quantize edge normal to four dir, if $M(x, y)$ smaller than either neighbour in dir, suppress, else keep

Def Thresholding Via thresholds $\theta_{\text{high}} \div \theta_{\text{low}} \in [2, 3]$ (typ)

1.3.1 Line Fitting

Generally challenging, missing information

Def RANSAC Random Sample Consensus:

1. Rand. sel. *seed group* of p. w/ base transf. est.
2. Compute transformation from seed group
3. Find *inliers* to this transformation
4. If inliers sufficiently large, recompute least-squares estimate of transformation on all inliers

Sample count: w frac of inliers, n points define hypothesis ($n = 2$ for lines), k samples chosen. Prob. sample of n points correct: w^n , prob all k samples fail: $(1 - w^n)^k$, i.e. **choose k high to keep below failure rate**

Def Hough Transform Voting technique, main idea:

1. Vote for all possible lines on which edge could lie
2. Look for line candidates that get many votes
3. Noise features votes should be inconsistent